Randomized Algorithms

CSCI-UA 480.73

Yao Xiao

Spring 2023

Contents

0 Introduction **1**

1 Repetition **2**

1.1 Verifying Polynomial Identities . . . 2

1.2 Verifying Matrix Multiplication . . . 3

1.3 Randomized Min-Cut . . . 3

2 Discrete Random Variables **5**

2.1 The Expected Runtime of Randomized QuickSort . . . 6

2.2 Coupon Collector's Problem . . . 7

2.3 Stable Matching . . . 8

2.4 String Matching (Fingerprinting) . . . 9

3 Randomized Binary Search Trees **12**

3.1 Treaps . . . 12

3.2 Skip Lists . . . 15

4 Moments and Deviations **17**

4.1 Markov's and Chebyshev's Inequalities . . . 17

1/23 Lecture

Instructor. Richard Cole, `cole@cs.nyu.edu`.

Class time. Mondays & Wednesdays 9:30–10:45am, 317 WWH.

Office hours. Mondays 4:00–5:00pm; also by appointment, 417 WWH.

0 Introduction

Example 0.1. Some examples of randomized algorithms versus deterministic algorithms:

- Quicksort vs. Merge Sort or Heap Sort
- Hashing vs. Balanced Trees
- Bloom Filters vs. Bit Arrays

- Stochastic Gradient Descent vs. Deterministic Gradient Descent (not covered but expected in Machine Learning or Scientific Computing)

1 Repetition

1.1 Verifying Polynomial Identities

Consider the polynomial identity

$$\det \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^{n-1} \end{bmatrix} = \prod_{1 \le i \le j \le n} (x_i - x_j). \tag{1}$$

Is this true for $n = 100$?

Problem. Given polynomials F and G, check if F and G are identical. That is, check if $H = F - G$ is identically zero. Suppose H is a polynomial of degree d.

Solution. Choose $r \in [1, 100d]$ uniformly at random, then evaluate $H(r)$. Note that uniform random number generation is assumed to run constant time. If $H(r) \neq 0$, then we can answer H is not identically zero. However, if $H(r) = 0$, the answer H is identically zero may be wrong. The probability that the answer is wrong is $\le \frac{d}{100d} = \frac{1}{100}$, since a non-identically-zero polynomial of degree d can have at most d real roots.

However, what if we want a lower error probability? We can run the algorithm above multiple times. If we run the algorithm k times, and each time $H(r_i) = 0$, then the probability of giving a wrong answer is $\le \left(\frac{1}{100}\right)^k$.

Probability Background

Randomized algorithms typically have a finite number (discrete set) of outcomes (based on the random choices). Write $\Omega = \{O_1, \cdots, O_m\}$ as the set of outcomes, *e.g.*, when rolling a die, $\Omega = \{1, 2, 3, 4, 5, 6\}$. For each outcome, we have a probability $0 \le \mathbb{P}[O_i] \le 1$, and

$$\sum_{i=1}^{m} \mathbb{P}[O_i] = 1. \tag{2}$$

Group outcomes into events as $E = \{O_{i_1}, \cdots, O_{i_k}\} \subseteq \Omega$, *e.g.*, a die lands on an even value. We use the set theory notations for events.

Lemma 1.1. If E_1 and E_2 are disjoint, then $\mathbb{P}[E_1 \cup E_2] = \mathbb{P}[E_1] + \mathbb{P}[E_2]$.

Lemma 1.2. $\mathbb{P}[E_1 \cup E_2] = \mathbb{P}[E_1] + \mathbb{P}[E_2] - \mathbb{P}[E_1 \cap E_2]$.

Lemma 1.3 (Inclusion-Exclusion Principle)**.** Given events $E_1, \cdots, E_n$, we have that

$$\mathbb{P}\left[\bigcup_{i=1}^{n} E_i\right] = \sum_{i=1}^{n} \mathbb{P}[E_i] - \sum_{i<j} \mathbb{P}[E_i \cap E_j] + \sum_{i<j<k} \mathbb{P}[E_i \cap E_j \cap E_k] - \cdots. \tag{3}$$

Events E and F are independent if and only if $\mathbb{P}[E \cap F] = \mathbb{P}[E] \cdot \mathbb{P}[F]$. Events $E_1, \cdots, E_k$ are mutually independent if and only if for every subset $I \subseteq \{1, \cdots, k\}$,

$$\mathbb{P}\left[\bigcap_{i \in I} E_i\right] = \prod_{i \in I} \mathbb{P}[E_i]. \tag{4}$$

1/25 Lecture

1.2 Verifying Matrix Multiplication

Problem. Test if $A \times B = C$, where A, B, and C are $n \times n$ matrices.

Solution. Let $r = (r_1, \cdots, r_n)$ an n-bit vector be chosen uniformly at random. Note that choosing $r \in \{0,1\}^n$ uniformly at random is equivalent to choosing $r_1, \cdots, r_n$ independently and uniformly at random (the **principle of deferred decisions**). Compute $s = Br$, $t = As$, and $w = Cr$. If $t = u$, we would answer true and we would answer false otherwise.

Now take $D = AB - C$ and suppose $D \neq 0$. Without loss of generality we assume that $d_{11} \neq 0$. Suppose $Dr = 0$, then we have that

$$d_{11}r_1 + \sum_{j=2}^{n} d_{1j}r_j = 0 \implies r_1 = -\frac{\sum_{j=2}^{n} d_{1j}r_j}{d_{11}}. \tag{5}$$

By the idea of the principle of deferred decisions, we can consider the right-hand side of (5) as already determined just before r_1 is chosen, leaving r_1 as random (or deferred). Since r_1 has two choices 0 or 1, the probability that $ABr = Cr$ is at most 1/2.

Probability Background

The conditional probability that event E occurs given that event F occurs (*e.g.*, probability that a die roll of 2 occurs given that the die roll is an even number) is

$$\mathbb{P}[E|F] = \frac{\mathbb{P}[E \cap F]}{\mathbb{P}[F]}. \tag{6}$$

Lemma 1.4. If E and F are independent events, then $\mathbb{P}[E \cap F] = \mathbb{P}[E]$.

Theorem 1.5 (Law of Total Probability)**.** Let $E_1, \cdots, E_n$ be mutually disjoint events in the sample space Ω, and suppose that $\bigcup_{i=1}^{n} E_i = \Omega$. Then

$$\mathbb{P}[B] = \sum_{i=1}^{n} \mathbb{P}[B \cap E_i] = \sum_{i=1}^{n} \mathbb{P}[B|E_i]\mathbb{P}[E_i]. \tag{7}$$

Now back to the problem, if $AB \neq C$, we can compute that

$$\begin{aligned}
\mathbb{P}[ABr = Cr] &= \sum_{(x_2,\cdots,x_n)\in\{0,1\}^{n-1}} \mathbb{P}[(ABr = Cr) \wedge ((r_1, \cdots, r_n) = (x_2, \cdots, x_n))] \\
&\leq \sum_{(x_2,\cdots,x_n)\in\{0,1\}^{n-1}} \mathbb{P}\left[\left(r_1 = -\frac{\sum_{j=2}^{n} d_{1j}r_j}{d_{11}}\right) \wedge ((r_1, \cdots, r_n) = (x_2, \cdots, x_n))\right] \\
&= \sum_{(x_2,\cdots,x_n)\in\{0,1\}^{n-1}} \mathbb{P}\left[r_1 = -\frac{\sum_{j=2}^{n} d_{1j}r_j}{d_{11}}\right] \cdot \mathbb{P}[(r_1, \cdots, r_n) = (x_2, \cdots, x_n)] \\
&\leq \sum_{(x_2,\cdots,x_n)\in\{0,1\}^{n-1}} \frac{1}{2} \cdot \mathbb{P}[(r_1, \cdots, r_n) = (x_2, \cdots, x_n)] = \frac{1}{2}.
\end{aligned} \tag{8}$$

Therefore, if $AB = C$ then the algorithm is always correct, and if $AB \neq C$ then the algorithm has 1/2 probability of going wrong. By k repetitions, assuming independence, the probability of getting an error would be reduced to 2^{-k}.

1.3 Randomized Min-Cut

Problem. A **cut-set** in a graph is a set of edges whose removal breaks the graph into two or more connected components. Given a graph G with n vertices, the min-cut problem is to find a minimum cardinality cut-set in G.

Solution. The main operation in the algorithm is **edge contraction**, as is shown in Figure 1. In contracting an edge (u, v), we merge the two vertices u and v into one vertex, eliminate all edges connecting u and v, and retain all other edges in the graph.

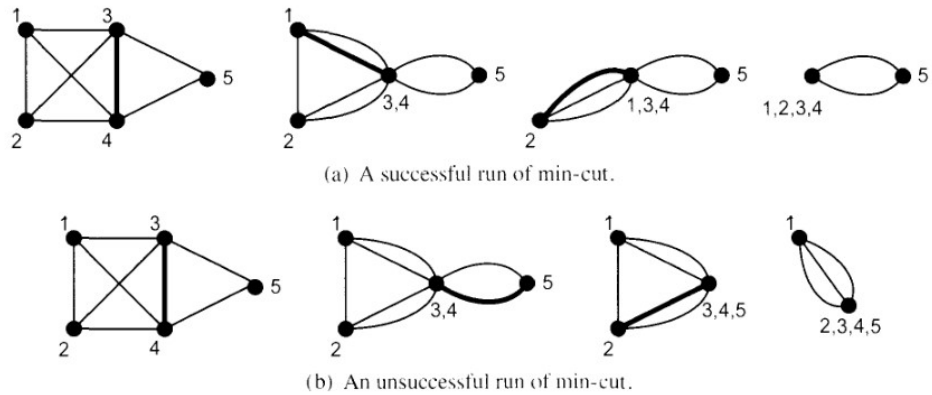

Figure 1: An example of two executions of min-cut in a graph with minimum cut-set of size 2.

The randomized algorithm consists of $n-2$ iterations. In each iteration, it chooses an edge uniformly at random from the remaining edges and perform edge contraction. After $n-2$ iterations, the graph will consist of only two vertices, and the algorithm outputs the set of edges connecting them.

1/30 Lecture

Theorem 1.6. The algorithm outputs a min-cut set with probability at least $\frac{2}{n(n-1)}$.

Proof. Let G be the graph with n vertices and m edges, and let C be a min-cut with $k = |C|$. Moreover, let E_i represent the event that the ith edge contracted is not in C, so $F_i = \bigcap_{h=1}^{i}$ will represent the event that none of the first i edges contracted are in C. Since the min-cut set C has k edges, each vertex must have degree at least k (otherwise we can separate a vertex of degree less than k from the rest of the vertices with fewer than k cuts). Therefore, the total number of edges in G is at least $nk/2$. Thus we have that

$$\mathbb{P}[F_1] = \mathbb{P}[E_1] \geq 1 - \frac{k}{nk/2} = 1 - \frac{2}{n}. \tag{9}$$

Similarly, suppose that F_1 happened, then

$$\mathbb{P}[E_2|F_1] \geq 1 - \frac{2}{n-1}. \tag{10}$$

Iterating this argument, we have that

$$\mathbb{P}[E_i|F_{i-1}] \geq 1 - \frac{2}{n-i+1}. \tag{11}$$

Since the randomized algorithm consists of $n-2$ iterations, we want to compute that

$$\begin{aligned}\mathbb{P}[F_{n-2}] &= \mathbb{P}[E_{n-2} \cap F_{n-3}] = \mathbb{P}[E_{n-2}|F_{n-3}] \cdot \mathbb{P}[F_{n-3}] = \cdots = \mathbb{P}[E_{n-2}|F_{n-3}] \cdots \mathbb{P}[E_2|F_1] \cdot \mathbb{P}[F_1] \\ &\geq \prod_{i=1}^{n-2}\left(1 - \frac{2}{n-i+1}\right) = \prod_{i=1}^{n-2} \frac{n-i-1}{n-i+1} = \frac{(n-2)(n-3)(n-4)\cdots 3 \cdot 2 \cdot 1}{n(n-1)(n-2)\cdots 5 \cdot 4 \cdot 3} = \frac{2}{n(n-1)}.\end{aligned} \tag{12}$$

Therefore, any min-cut of G can be found with probability at least $\frac{2}{n(n-1)}$. However, since we cannot know how many min-cut sets there are in G, so this bound is the best we can guarantee. $\square$

Using l repetitions, the probability of never finding a min-cut is at most

$$\left(1-\frac{2}{n(n-1)}\right)^l \leq \underbrace{\left(\exp\left(-\frac{2}{n(n-1)}\right)\right)^l}_{1-x\leq\exp(-x)} = \exp\left(-\frac{2l}{n(n-1)}\right). \tag{13}$$

By choosing $l = \frac{cn(n-1)\ln n}{2}$, this probability is at most $\exp(-c\ln n) = n^{-c}$.

Monty Hall Problem

Problem. Suppose you are on a game show, and you are given the choice of three doors: Behind one door is a car; behind the others, goats. You randomly pick a door, and the host will reveal another door with a goat inside. Then you are prompted to make a switch to the other door. Will this give you an advantage?

Solution. If one does not make the switch, he has only 1/3 probability of getting the car, since his first choice is independent of any proceeding activity. On the other hand, if one makes the switch, he will have 2/3 probability of getting the car. Consider a much more extreme case: Suppose there are a million doors with only one car, and the host reveals 999998 doors with goats after the first choice. The chance of picking a car in the first choice is only 1/1000000, and this probability will not raise because of revealing the other doors. Moreover, by opening the doors with goats, the host is almost telling the answer, so making a switch is indeed a wise choice.

2/1 Lecture

2 Discrete Random Variables

Probability Background

For example, X is the sum of the outcomes of the rolls of two dice, then for instance $X = 4$ corresponds to the outcomes $(1,3)$, $(2,2)$, and $(3,1)$. This is an example of a discrete random variables, which takes a finite or countable number of values.

Definition 2.1. Random variables X and Y are **independent** if and only if

$$\mathbb{P}[(X=x)\wedge(Y=y)] = \mathbb{P}[X=x]\cdot\mathbb{P}[Y=y], \qquad \forall x,y. \tag{14}$$

Definition 2.2. Random variables $X_1,\cdots,X_k$ are **mutually independent** if and only if for any subset $I \subseteq \{1,\cdots,k\}$ and values x_i, $i\in I$,

$$\mathbb{P}\left[\bigwedge_{i\in I}(X_i=x_i)\right] = \prod_{i\in I}\mathbb{P}[X_i=x_i]. \tag{15}$$

Definition 2.3. The **expectation** (average value) of a random variable X is

$$\mathbb{E}[X] = \sum_i i\cdot\mathbb{P}[X=i]. \tag{16}$$

For instance, let X be the outcome of one roll of a die, then we have that

$$\mathbb{E}[X] = \frac{1}{6}(1+2+3+4+5+6) = \frac{7}{2}. \tag{17}$$

As another example, let Y be the sum of the rolls of two dice, then we have that

$$\mathbb{E}[Y] = \frac{1}{36}\cdot 2 + \frac{2}{36}\cdot 3 + \frac{3}{36}\cdot 4 + \cdots + \frac{1}{36}\cdot 12 = 7. \tag{18}$$

Theorem 2.4 (Linearity of expectations)**.** Let $X_1,\cdots,X_n$ be random variables, then

$$\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i], \tag{19}$$

whether or not the random variables are (mutually) independent.

2.1 The Expected Runtime of Randomized QuickSort

Recall that the QuickSort algorithm can be represented as in Algorithm 1. For the sake of simplicity, we call the

Algorithm 1 QuickSort

Require: an array S of n distinct elements over a totally sorted universe.

1: choose an element x of S as the pivot;
2: compare every other element of S to x in order to divide the other elements into two subarrays, such that S_1 has all the elements of S that are less than x, and S_2 has all the elements of S that are greater than x;
3: run QuickSort (Algorithm 1) on S_1 and S_2, respectively;
4: **return** the concatenated array of S_1, $\{x\}$, and S_2;

elements of S as $z_1, \cdots, z_n$, where z_i represents the ith smallest element of S. In order to analyze the runtime, we want to count the number of comparisons. Since QuickSort will compare elements only to the pivots, and a pivot will be excluded from any further recursive calls, each pair of elements in S must be compared at most once. Therefore, *indicator random variables* can be used to count the number of comparisons. Let X be the total number of comparisons for sorting S and let X_{ij} represent the number of comparisons between z_i and z_j, then we have that

$$X = \sum_{i<j} X_{ij}. \tag{20}$$

Moreover, let E_{ij} be the event that z_i is compared with z_j. By the linearity of expectations, we can deduce that

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i<j} X_{ij}\right] = \sum_{i<j} \mathbb{E}[X_{ij}] = \sum_{i<j} (0 \cdot \mathbb{P}[\overline{E_{ij}}] + 1 \cdot \mathbb{P}[E_{ij}]) = \sum_{i<j} \mathbb{P}[E_{ij}]. \tag{21}$$

Therefore, it suffices to compute $\mathbb{P}[E_{ij}]$, *i.e.*, the probability that z_i and z_j are compared during the sorting. Consider the elements $z_i, z_{i+1}, \cdots, z_{j-1}, z_j$. If any of the elements $z_{i+1}, \cdots, z_{j-1}$ has been chosen as a pivot, then z_i and z_j must have been partitioned into different subarrays but QuickSort and will never be compared. Therefore, the only chance that z_i is compared with z_j is that either z_i or z_j is the first element to be chosen as a pivot among the elements $z_i, z_{i+1}, \cdots, z_{j-1}, z_j$ during the sorting. Therefore, we are choosing one of the two elements among a total of $j - i + 1$ elements, and thus

$$\mathbb{P}[E_{ij}] = \frac{2}{j-i+1}, \qquad \forall 1 \le i < j \le n. \tag{22}$$

Therefore, (21) can be written as

$$\mathbb{E}[X] = \sum_{i<j} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \underbrace{\sum_{l=1}^{n-i} \frac{2}{l+1}}_{l=j-i} < \sum_{i=1}^{n-1} \sum_{l=1}^{n-i} \frac{2}{l} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n). \tag{23}$$

Since each comparison takes constant time, the runtime of making comparisons is $O(n \log n)$. Furthermore, this dominates the linear time required for partitioning the elements, thus the expected runtime of the randomized QuickSort is $O(n \log n)$.

2/6 Lecture

Skipped due to technical issues.

2/8 Lecture

Probability Background

Suppose that we run an experiment that succeeds with probability p and fails with probability $1 - p$. Let Y be a random variable such that $Y = 1$ if the experiment succeeds and $Y = 0$ otherwise. The variable Y is called a

Bernoulli random variable or an **indicator random variable**. Note that

$$\mathbb{E}[Y] = p \cdot 1 + (1-p) \cdot 0 = p. \tag{24}$$

Now, if we perform the experiment above for n times, and let X be a random variable representing the number of successes, then

$$\mathbb{P}[X = j] = \binom{n}{j} p^j (1-p)^{n-j}, \tag{25}$$

which defines a **binomial random variable**. Note that $X = \sum_{j=1}^{n} Y_j$ if each Y_j is the Bernoulli random variable for the jth experiment, and it follows from the linearity of expectation that $\mathbb{E}[X] = np$.

Now suppose that we perform the experiment above until the first success, and denote by a random variable Z the number of experiments we perform. Then

$$\mathbb{P}[Z = n] = (1-p)^{n-1} p. \tag{26}$$

This is called a **geometric random variable**. Note that it satisfies the **memoryless** property, that is,

$$\mathbb{P}[Z = n+k | Z > k] = \frac{\mathbb{P}[(Z = n+k) \cap (Z > k)]}{\mathbb{P}[Z > k]} = \frac{(1-p)^{n+k-1} p}{\sum_{i=k+1}^{\infty} (1-p)^{i-1} p} = \frac{(1-p)^{n+k-1} p}{(1-p)^k} = (1-p)^n 0 = \mathbb{P}[Z = n]. \tag{27}$$

Moreover, we can compute the expectation of the geometric random variable Z as

$$\mathbb{E}[Z] = \sum_{j=1}^{\infty} j \mathbb{P}[Z = j] = \sum_{j=1}^{\infty} j (1-p)^{j-1} p = p \sum_{j=1}^{\infty} j (1-p)^{j-1} =: pI. \tag{28}$$

Note that

$$I = \sum_{j=1}^{\infty} j (1-p)^{j-1} = 1 + 2(1-p) + 3(1-p)^2 + 4(1-p)^3 + \cdots, \tag{29}$$

$$(1-p)I = \sum_{j=1}^{\infty} j (1-p)^{j} = (1-p) + 2(1-p)^2 + 3(1-p)^3 + \cdots, \tag{30}$$

so we can deduce that

$$\mathbb{E}[Z] = pI = I - (1-p)I = 1 + (1-p) + (1-p)^2 + (1-p)^3 + \cdots = \frac{1}{1-(1-p)} = \frac{1}{p}. \tag{31}$$

2.2 Coupon Collector's Problem

Problem. Suppose that each box of cereal contains one of n different coupons. Once you obtain one of every type of coupon, you can send in for a prize. Assuming that the coupon in each box is chosen independently and uniformly at random from the n possibilities, how many boxes of cereal must you buy before you can send in for the prize?

Solution. Let X be the number of boxes one must buy before obtaining all n types of coupons. Moreover, let X_i denote the number of purchases to obtain i types of coupons after having obtained $i-1$ types of coupons. Then we have that

$$X = \sum_{i=1}^{n} X_i. \tag{32}$$

Note that after having obtained $i-1$ types of coupons, if we want to further obtain one more type of coupon, each purchase will be of success probability $\frac{n-i+1}{n}$. Therefore, each X_i is a geometric random variable with success probability $\frac{n-i+1}{n}$, and thus by the linearity of expectation, we can obtain that

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{n}{n-i+1} = n \underbrace{\sum_{j=1}^{n} \frac{1}{j}}_{j=n-i+1} =: nH(n). \tag{33}$$

Note that $H(n) = \sum_{i=1}^{n} \frac{1}{i}$ is called the **harmonic number**, which we can approximate by

$$\sum_{i=1}^{n} \frac{1}{i} > \int_{1}^{n+1} \frac{1}{x} dx = \ln x \Big|_{x=1}^{n+1} = \ln(n+1), \tag{34}$$

$$\sum_{i=1}^{n} \frac{1}{i} = 1 + \sum_{i=2}^{n} \frac{1}{i} < 1 + \int_{1}^{n} \frac{1}{x} dx = 1 + \ln x \Big|_{x=1}^{n} = 1 + \ln n, \tag{35}$$

as is demonstrated in Figure 2.

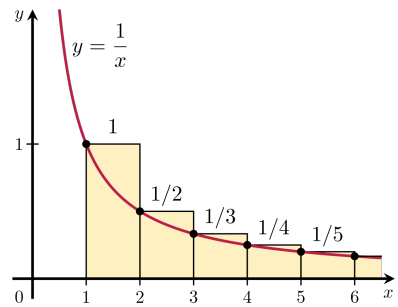

Figure 2: The lower bound for the harmonic number by integration. Note that this is obtained by taking the left boundary value for each bar. If we alternatively take the right boundary value for each bar, we will obtain an upper bound.

Back to the problem, we can now conclude that

$$n \ln(n+1) < \mathbb{E}[X] < n + n \ln n \implies \mathbb{E}[X] = \Theta(n \ln n). \tag{36}$$

2/13 Lecture

2.3 Stable Matching

Problem. Suppose there are n doctors and n hospitals. Each doctor has an ordering of preference for the hospitals, and each hospital also has an ordering of preference for the doctors. We say that (d, H) is a **blocking pair** if both d and H prefer each other to their current matches. A stable matching is a matching with no blocking pairs.

Solution. We propose the deferred acceptance algorithm as shown in Algorithm 2, which runs $O(n^2)$ time.

Algorithm 2 Deferred acceptance (doctor proposing)

```
while some doctor d is unmatched do
    d proposes to the next hospital H on her list;
    if H is unmatched or H prefers d to its current match d' then
        H is tentatively matched to d and d' is left unmatched;
    end if
end while
```

Lemma 2.5. Every doctor gets a match in Algorithm 2.

Proof. Once tentatively matched, a hospital remains matched. Therefore, if d is unmatched, at most $n-1$ hospitals have ever been matched. In the worst case, the nth hospital will be proposed to by d eventually, so that d will be matched. By arbitrariness of d we complete the proof. □

Lemma 2.6. The matching in Algorithm 2 is stable.

Proof. Each hospital's match only improves as the algorithm proceeds. In other words, each hospital is matched to the best proposal it received. Moreover, observe that each doctor gets the best match among those proposals that are not rejected. Now assume for contradiction that (d, H) forms a blocking pair, then d and H prefer each other to their current matches, say d to H' and H to d'. However, H' must be the best match for d among those that does not reject d. That is to say, if d prefers H over H', then H must have rejected d. Given that each hospital is matched to the best proposal it received, H must prefer d' over d. This leads to a contradiction, implying that their is no blocking pair produced by the algorithm. The matching is thus stable and the proof is done. □

Randomized input. Assume that the same problem statement as above, with each preference list a uniform draw over all permutations of the other side.

Theorem 2.7. In the uniform random preference model, the expected number of proposals using Algorithm 2 is $O(n \log n)$.

Proof. We use the principle of deferred decisions. In other words, we do not assume that the random preference lists are chosen in advance. Instead, we assume that each doctor does not know his preference list and each time he makes a proposal, he picks a hospital uniformly at random. This problem then immediately resembles the coupon collector's problem, observing that Algorithm 2 terminates once all the hospitals have received at least one proposal (since once a hospital receives a proposal, it will stay matched though its match may change, and once all the hospitals have received a proposal, this means all the hospitals are matched, thus all the doctors must have been matched as well). In other words, the doctors randomly proposes hospitals until all the n hospitals have received a proposal. We know that the expected number would then be $nH(n) \approx \Theta(n \ln n)$. However, this is only an upper bound. Doctors actually "memorize" which hospitals have rejected them so that they will not propose to the same hospitals again, instead of proposing completely randomly. Therefore, the total number of proposals could have been smaller, meaning that the expected number of proposals using Algorithm 2 is $O(n \ln n)$, and the proof is complete. □

2.4 String Matching (Fingerprinting)

Problem. Given a text $t = t_1 t_2 \cdots t_m$ and a pattern $p = p_1 p_2 \cdots p_n$ (assuming $m > n$), we want to find the occurrences of p in t. A basic example of string matching is when the pattern and the searched text are arrays of elements of an alphabet (finite set), which can be a human language alphabet (A through Z), a binary alphabet $\{0, 1\}$, or a DNA alphabet (A,C,G,T) in bioinformatics, etc. For a general notation, we denote the alphabet as $\Sigma = \{0, 1, \cdots, s-1\}$ so that each character is treated as a number.

Solution. The naive solution would be to create a window of length n and move through t, comparing the substring in the window to p in each iteration. This would be $O(m)$ iterations and $O(n)$ time for each iteration, thus requiring $O(mn)$ total runtime. To improve the efficiency, we propose the **Karp-Rabin algorithm**, which compares the "hash" values of the substrings and p, thus reducing each iteration to constant time. More specifically, calculating the initial hash values will take $O(n)$ time, but subsequent updating will be $O(1)$ for each iteration among the total $O(m)$ of them. This gives a better $O(n+m)$ runtime algorithm, but this can give error. To see this, we will represent p as a number, called $\texttt{val}(p)$, such that

$$\texttt{val}(p) = p_1 s^{n-1} + p_2 s^{n-2} + \cdots + p_{n-1} s + p_n \mod q, \tag{37}$$

where q is a suitable prime number that we will specify later. We also assume that each arithmetic modulus operation takes $O(1)$ time. We represent each of the substrings t^{i+1} (of length n), $0 \le i \le n-m$, using the same encoding as

$$\texttt{val}(t^{i+1}) = t_{i+1} s^{n-1} + t_{i+2} s^{n-2} + \cdots + t_{i+n-1} s + t_{i+n} \mod q. \tag{38}$$

Clearly if $t^{i+1} = p$, then $\texttt{val}(t^{i+1}) = \texttt{val}(p)$, but the reverse is not true. This can lead to a non-match being reported as a match, so the idea of the algorithm is to choose q so that the probability of this event is small. To do this, let us examine the situation when $t^{i+1} \neq p$ yet $\texttt{val}(t^{i+1}) = \texttt{val}(p)$. In this situation, we have that $\texttt{val}(p) - \texttt{val}(t^{i+1}) \equiv 0 \mod q$, implying that $\texttt{val}(p) - \texttt{val}(t^{i+1})$ is an integer multiple of q. Now note that $\texttt{val}(p), \texttt{val}(t^{i+1}) < s^n$, since they are at most

$$\sum_{i=1}^{n} (s-1) s^{n-i} = \sum_{i=1}^{n} s^{n-i+1} - \sum_{i=1}^{n} s^{n-i} = \sum_{i=0}^{n-1} s^{n-i} - \sum_{i=1}^{n} s^{n-i} = s^n - 1 < s^n. \tag{39}$$

As a result, $|\texttt{val}(p) - \texttt{val}(t^{i+1})| < s^n$. Suppose that $q \geq s$, then there are at most n distinct primes q for which $\texttt{val}(p) - \texttt{val}(t^{i+1})$ is an integer multiple of q. Otherwise, $\texttt{val}(p) - \texttt{val}(t^{i+1})$ can be expressed as an integer multiple of the product of more than n distinct primes that are $\geq s$, thus exceeding s^n and leading to a contradiction.

Given the previous observations, we will choose q uniformly at random from a large set of primes, so that there is only a small chance that q is one of the at most n primes that divide $\texttt{val}(p) - \texttt{val}(t^{i+1})$ exactly. In particular we choose q from a range $[v, 2v)$ to be specified shortly. This is based on a known fact that the number of primes in this range is at least $\frac{v}{2 \ln v}$ for $v \geq 61000$ (this is a loose bound). The algorithm to choose a prime selects an integer in this range uniformly at random and tests it for primality, and keeps going until a prime number is found. In expectation, $2 \ln v$ tests would suffice. As we will see later, each primality test can be done by a linear time randomized algorithm which has a small probability of reporting a composite as a prime (in which case the Karp-Rabin algorithm may report incorrect results). We will keep this probability to at most $\frac{1}{2n^c}$, where $c > 0$ is an arbitrary constant.

We would like to achieve a probability of at most $\frac{1}{2m^c}$ of having an incorrect answer, assuming that the q provided by the primality test is actually a prime number. We need to test $n - m + 1$ alignments of p with substrings of t, so it suffices to have a probability of an incorrect answer on any given test of $\frac{1}{2m^{c+1}}$. To this end, we need that the number of primes in the interval $[v, 2v)$ be at least $n \cdot 2m^{c+1}$, since there are at most n "bad" choices of q. Therefore, it suffices to have that

$$\frac{v}{2 \ln v} \geq 2nm^{c+1}. \tag{40}$$

Choosing $v = 8(c+2)nm^{c+1} \ln m$ would suffice (JUST TRUST THE PROFESSOR), and recall that we also want $v \geq s$ (though in practice this condition is satisfied automatically most of the times).

2/15 Lecture

Miller-Rabin Primality Test

We propose the Miller-Rabin primality test with probability of error at most 2^{-k} for some constant k, that is used in the previous algorithm. Suppose that m is the integer being tested (with $n = \log m$ bits).

Lemma 2.8. Consider the values $f(x) = x^{(n-1)/2} \mod n$ for numbers x coprime to n.

(1) If m is a prime number, then there are exactly $(n-1)/2$ numbers x for which $f(x) = -1$, and $(n-1)/2$ numbers x for which $f(x) = 1$.

(2) If m is an odd-valued composite number which has two distinct prime factors, then either $f(x) = 1$ for all of these x or $f(x) \neq \pm 1$ for at least half of these x.

Proof. This lemma can be shown by elementary arguments, but the proof is long which will be neglected here. □

In order to apply this lemma for primality testing, we need three algorithms: one for exponentiation in order to compute $f(x)$ as in the lemma, one for finding the greatest common divisor, and one to test if $m = x^i$ for some integers $x \geq 2$ and $i \geq 2$.

The exponentiation algorithm. To compute $x^y \mod n$ we proceed as follows.

- By repeated squaring, compute the following values modulus m: $x, x^2, x^4, \cdots, x^k$, where $2^k \leq y < 2^{k+1}$. As all results are computed modulus n, if we use long multiplication and division, then each squaring takes $O(n^2)$ time, and so $O(n^3)$ time is needed overall. WHY? WHAT IS THE RELATION BETWEEN k AND n?
- Suppose that $y = 2^{i_1} + 2^{i_2} + \cdots + 2^{i_j}$, where $0 \leq i_1 < i_2 < \cdots < i_j \leq k$, then we multiply the terms $x^{2^{i_1}}, x^{2^{i_2}}, \cdots, x^{2^{i_j}}$ together to obtain the result. This takes a further $O(n^3)$ time. WHY?

Algorithm 3 GCD

```
c ← a mod b;
if c ≠ 0 then
    return GCD(b, c);
else
    return b;
end if
```

The greatest common divisor algorithm. We propose an algorithm to compute $\gcd(a, b)$ for $a > b$, as is described in Algorithm 3.

Lemma 2.9. Algorithm 3 runs in $O(n^3)$ time on n-bit input integers.

Proof. We observe that $c < a/2$. Indeed, if $b > a/2$, then $c = a \mod b = a - b < a/2$. Otherwise if $b \leq a/2$, then $c < b \leq a/2$. If $c \geq 1$, then by a similar reasoning $\gcd(b, c) < b/2$. Therefore, we can see that in at most two iterations, the arguments passed to the algorithm halve in value, and thus the algorithm proceeds for at most $2n$ iterations. Each iteration entails a modulus operations which takes $O(n^2)$ time for long integers, so the total runtime would be $O(n^3)$, as desired. □

The power testing algorithm. This algorithm is intended to test if $m = x^i$ for some integers $x \geq 2$ and $i \geq 2$. It seeks, for each i in turn, to identify the value x such that $(x-1)^i < m \leq x^i$, by means of binary search over the possible values of x. Since m is an n-bit number, the values of i that need to be tested are $2 \leq i < n$. A test proceeds as follows. It will iteratively improve an estimate y of x such that $y = w \cdot 2^h$ for some integers w and h, and we will maintain the property that $y < x \leq y + 2^h - 1$. In other words, y consists of the leading bits of x. Each iteration of our algorithm will add one more bit to the estimate, so the algorithm for value i will use n/i iterations. The algorithm is described as in Algorithm 4.

Algorithm 4 IsPower

```
y ← 1, h ← ⌊n/i⌋;
while h > 0 do
    z ← (2y + 1)2^(h−1), w ← z^i;
    if w = m then
        return true;
    else if w < m then
        y ← z, h ← h − 1;
    else
        y ← z − 1, h ← h − 1;
    end if
end while
```

Lemma 2.10. Algorithm 4 runs in time $O(n^3/i)$, and testing whether m is of the form x^i for some pair of integers $x, i \geq 2$, takes time $O(n^3 \log n)$.

Proof. If we use repeated squaring to compute w, assuming that we multiple the numbers in order of increasing (doubling) length, this will take $O(n^2)$ time. We have to repeat this n/i times, giving an overall runtime of $O(n^3/i)$. Since we need to repeat this algorithm for increasing values of $i \leq n$, the overall runtime would sum to $O(n^3 H(n)) = O(n^3 \log n)$, so the proof is complete. □

Now we can propose the **Miller-Rabin** primality test algorithm, as is described in Algorithm 5.

Lemma 2.11. The Miller-Rabin algorithm has an error probability of at most 2^{-k}.

Proof. The first two tests identify composites that are even integers or integer powers of some smaller integer, so the numbers that remain to be tested are either prime or composites with at least two distinct prime factors. For such numbers, if they are prime, then $r_i = \pm 1$ for all $1 \leq i \leq k$, and each value have equal probability of occurring. Thus in this case, the probability that $r_i = 1$ for all $1 \leq i \leq k$ is 2^{-k}, which means that the probability of reporting a

Algorithm 5 Miller-Rabin primality test

```
if m is even or m = x^i for some integers x, i ≥ 2 then
    return composite;
end if
select k integers b_1, ···, b_k in the range [1, n − 1] uniformly at random;
if gcd(n, b_i) > 1 for some 1 ≤ i ≤ k then
    return composite;
end if
compute r_i = b_i^((n−1)/2) mod n for 1 ≤ i ≤ k;
if r_i ≠ ±1 for some 1 ≤ i ≤ k then
    return composite;
else if r_i = 1 for all 1 ≤ i ≤ k then
    return composite;
else
    return prime;
end if
```

prime as a composite is 2^{-k}. Likewise, for such numbers, if they are composite, then either every r_i with b_i coprime to n equals 1, or at least half of these $r_i \neq \pm 1$. In the first case, the number is identified as composite for sure; in the second case, it is misidentified as a prime with probability at most 2^{-k}. □

Lemma 2.12. The Miller-Rabin algorithm runs $O(kn^3 + n^3 \log n)$ time.

Proof. The first testing condition runs the power testing algorithm (Algorithm 4), which takes $O(n^3 \log n)$ time. Then, selecting k integers uniformly at random takes $O(k)$ time. The second testing condition runs the greatest common divisor algorithm (Algorithm 3) for k times, which is overall $O(kn^3)$ runtime. Then, computing r_i requires running the exponentiation algorithm k times, which again runs overall $O(kn^3)$ time. The final judging can be done in the meantime when computing r_i. Therefore, the Miller-Rabin algorithm takes $O(kn^3 + n^3 \log n)$ time in total. □

2/22 Lecture

3 Randomized Binary Search Trees

We are going to consider some randomized alternatives to balanced binary search tree structures such as AVL trees, red-black trees, B-trees, or splay trees, which are arguably simpler than any of these deterministic structures.

3.1 Treaps

The term "treap" is a combination of "tree" and "heap". Recall that binary search trees require the values in the left subtree to be less than the value of the current node, and the values in the right subtree to be greater than the value of the current node. Also recall that (min-)heaps require the parents to have lower priorities than the children. An example is shown as in Figure 3.

Insertion. We start by using the standard binary search tree insertion algorithm to insert a new node at the bottom of the tree. To this end, the search keys still form a search tree, but the priorities may no longer form a heap. For instance, if we want to insert $\mathbf{S}_{-1}$ into the treap in Figure 3. It will be inserted as the right child of $\mathbf{R}_5$. However, $5 > 0$ so the heap property is violated. In this case, recall the **rotation** operation in binary search trees, as is shown in Figure 4. If the priority of the yellow node is greater than the green node, then we perform the left rotation (from left to right in Figure 4) if necessary. Otherwise, we perform the right rotation (from right to left in Figure 4) if necessary.

Note that the priority violation may not only happen between the new node and its parent. Even if we perform a left rotation between $\mathbf{R}_5$ and $\mathbf{S}_{-1}$, there will still be a violation between $\mathbf{T}_3$ and $\mathbf{S}_{-1}$. Therefore, we need to deal with all the possible priority violations between the new node and its ancestors (any other violation is not possible). An illustration is shown as in Figure 5.

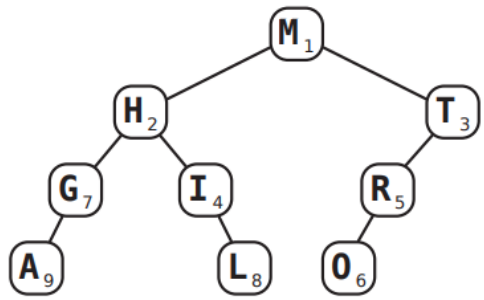

Figure 3: An example of a treap. Letters represent the search keys and numbers represent the priorities.

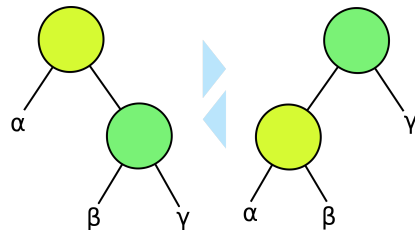

Figure 4: Rotations of binary search trees.

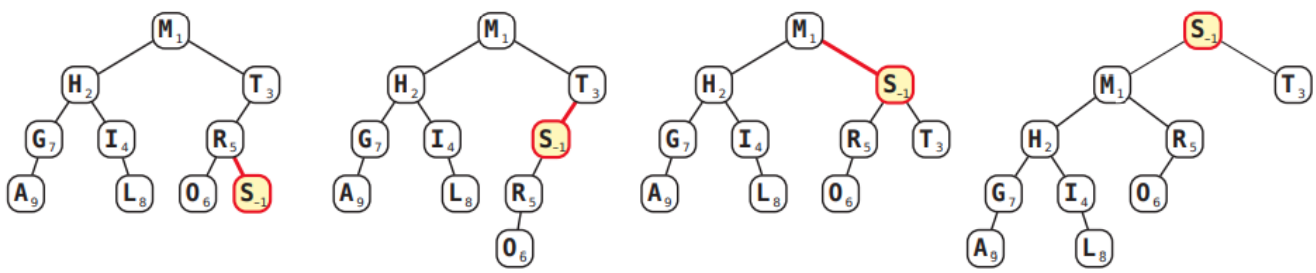

Figure 5: An example of inserting a new node in a treap.

Deletion. To delete a node, we just run the insertion algorithm backward in time. Suppose we want to delete node z. As long as z is not a leaf, perform a rotation at the child of z with smaller priority. This moves z down a level and its smaller-priority child up a level. The choice of which child to rotate preserves the heap property everywhere except at z. When z becomes a leaf, chop it off.

Bound expected depth. We will use the following notations.

Notation	Interpretation
x_k	the node/item with the kth smallest key
$i \uparrow k$	i being a proper ancestor of k
[`Property`]	1 if `Property` is true and 0 otherwise
$\mathrm{Depth}(x_k)$	$\sum_{i=1}^{n}[i \uparrow k]$
$X(i,k)$	either $\{x_i, x_{i+1}, \cdots, x_k\}$ or $\{x_k, x_{k+1}, \cdots, x_i\}$

Note that

$$\mathbb{E}[\mathrm{Depth}(x_k)] = \sum_{i=1}^{n} \mathbb{E}[[i \uparrow k]] = \sum_{i=1}^{n} \mathbb{P}[i \uparrow k], \tag{41}$$

so it suffices to consider the probability that some node is a proper ancestor of some other node.

Lemma 3.1. For all $i \neq k$, we have $i \uparrow k$ if and only if x_i has the smallest priority among all nodes in $X(i,k)$.

Proof. There are four cases to consider.

- If x_i is the root, then $i \uparrow k$ for all k, so by definition it must have the smallest priority in $X(i,k)$.
- If x_k is the root, then $k \uparrow i$ for all i. Therefore, x_i does not have the smallest priority in $X(i,k)$ since x_k does.
- Suppose some other node x_j is the root, and x_i and x_k are in different subtrees, then either $i < j < k$ or $k < j < i$. In any of these two cases, $x_j \in X(i,k)$, so both $i \not\uparrow k$ and $k \not\uparrow i$. Moreover, x_i does not have the smallest priority in $X(i,k)$ since x_j does.
- Suppose some other node x_j is the root, and x_i and x_k are in the same subtree. Note that the subtree is just a smaller treap, so the lemma would follow by induction.

Combining the cases above, the proof is complete. □

Since each node in $X(i,k)$ is equally likely to have the smallest priority, it immediately follows that

$$\mathbb{P}[i \uparrow k] = \frac{[i \uparrow k]}{|k-i|+1} = \begin{cases} \dfrac{1}{k-i+1}, & \text{if } i < k, \\ 0, & \text{if } i = k, \\ \dfrac{1}{i-k+1}, & \text{if } i > k. \end{cases} \tag{42}$$

Plugging into the expression of the expected depth, we can thus deduce that

$$\begin{aligned} \mathbb{E}[\mathrm{Depth}(x_k)] &= \sum_{i=1}^{n} \mathbb{P}[i \uparrow k] = \sum_{i=1}^{k-1} \frac{1}{k-i+1} + \sum_{i=k+1}^{n} \frac{1}{i-k+1} = \sum_{j=2}^{k} \frac{1}{j} + \sum_{j=2}^{n-k+1} \frac{1}{j} \\ &= H(k) + H(n-k+1) - 2 < \ln k + \ln(n-k+1) - 2 < 2\ln n - 2 = O(\ln n). \end{aligned} \tag{43}$$

Now let us review the cost of each operation in the treap.

- **Search:** A successful search for key k takes $O(\mathrm{Depth}(x_k))$ time. Let x_k^- and x_k^+ be the inorder predecessor and successor of x_k, respectively, then the time for an unsuccessful search is either $O(\mathrm{Depth}(x_k^-))$ or $O(\mathrm{Depth}(x_k^+))$.
- **Insertion/Deletion:** Inserting a new node with key k takes either $O(\mathrm{Depth}(x_k^-))$ or $O(\mathrm{Depth}(x_k^+))$ time. Deletion is just insertion in reverse.
- **Split/Join:** Splitting a treap at pivot value k takes either $O(\mathrm{Depth}(x_k^-))$ or $O(\mathrm{Depth}(x_k^+))$ time, since it costs the same as inserting a new dummy root with search key k and priority $-\infty$. Join is just split in reverse.

Therefore, given that $\mathbb{E}[\mathrm{Depth}(x_k)] = O(\ln n)$, all the treap operations mentioned above in an n-node treap will take $O(\ln n)$ expected time.

Randomized Quicksort Revisited

Here we propose a new way to describe the randomized Quicksort algorithm as in Algorithm 6.

Algorithm 6 Randomized QuickSort (treap version)

Require: an array S of n distinct elements over a totally sorted universe.

```
T ← an empty binary search tree;
insert the keys in S into T in random order;
return the inorder sequence of keys in T;
```

Since each key is inserted in $O(\ln n)$ expected time, the expected runtime of this randomized Quicksort algorithm will run in $O(n \ln n)$ expected time. This is a kind of Quicksort since roots are like pivots, and the two subtrees are like to two partitioned parts.

2/27 Lecture

3.2 Skip Lists

A skip list is just a sorted linked list with some random shortcuts. For each item in the linked list, we duplicate it with a 1/2 probability, string together all the duplicates into a second sorted linked list, and point each duplicate back to the original. We then repeat this step recursively, until there is no node left. An example of a skip list is shown as in Figure 6. When we insert an item into a skip list, we flip a coin and make a copy at each head, until we finally flip a tail. If the corresponding level does not exist, we create a new level. Deletion removes the key in every level that it exists. Searching is also simple: we start at the leftmost node in the highest level, scan through each level as far as we can without passing the target value x, and then proceed down to the next level. The search ends when we either reach a node with search key x or fail to find x on the lowest level.

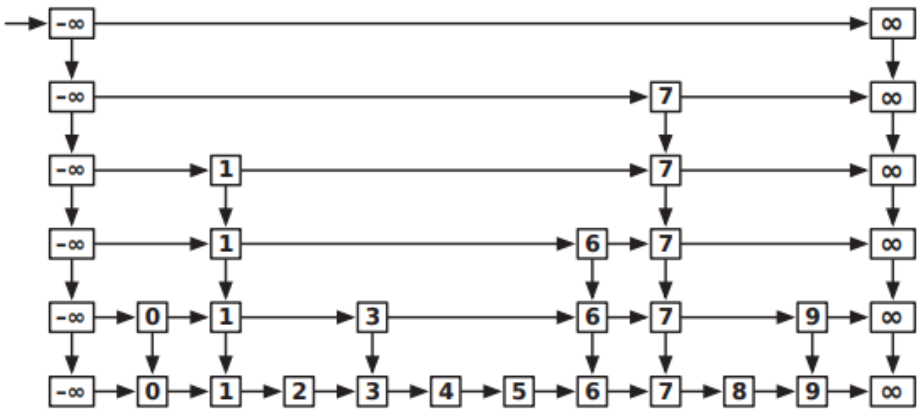

Figure 6: An example of the skip list data structure.

Number of levels. Assume that the keys in the skip list are the integers from 1 to n. Let $L(x)$ denote the number of levels of the skip list that contains the certain search key x, not counting the bottom level. Also assume that we duplicate x if a coin flip shows the head. Then we can compute the expectation of $L(x)$ recursively, such that

$$\mathbb{E}[L(x)] = \underbrace{\frac{1}{2} \cdot 0}_{\text{flip tail}} + \underbrace{\frac{1}{2} \cdot (1 + \mathbb{E}[L(x)])}_{\text{flip head}} \implies \mathbb{E}[L(x)] = 1. \tag{44}$$

Moreover, we need a bound on the number of levels $L = \max_x L(x)$. Note that in order for a search key to appear on level l, it must have flipped l heads in a row when it was inserted, so that

$$\mathbb{P}[L(x) \geq l] = \frac{1}{2^l}, \qquad \forall x. \tag{45}$$

The skip list has at least l levels if and only if $L(x) \geq l$ for at least one of the n search keys. Therefore,

$$\mathbb{P}[L \geq l] = \mathbb{P}\left[\bigwedge_x (L(x) \geq l)\right] \leq \sum_x \mathbb{P}[L(x) \geq l] = \frac{n}{2^l}. \tag{46}$$

When $l \leq \log n$, this bound is trivial. However, for any constant $c > 1$, we have a strong upper bound

$$\mathbb{P}[L \geq c \log n] \leq \frac{n}{2^{c \log n}} = \frac{n}{n^c} = \frac{1}{n^{c-1}}, \tag{47}$$

which implies that with high probability, a skip list has $O(\log n)$ levels. Moreover, we can derive an upper bound for the expectation of L, such that

$$\mathbb{E}[L] = \sum_{l \geq 0} l \cdot \mathbb{P}[L = l] = \sum_{l \geq 1} \mathbb{P}[L \geq l] \leq \sum_{l=1}^{\lfloor \log n \rfloor} 1 + \sum_{l=\lfloor \log n \rfloor + 1}^{\infty} \frac{n}{2^l} = \lfloor \log n \rfloor + \frac{\frac{n}{2^{\lfloor \log n \rfloor + 1}}}{1 - \frac{1}{2}} \leq \lfloor \log n \rfloor + 2. \tag{48}$$

Therefore, in expectation, a skip list has at most two more levels than an ideal version where each level contains exactly half the nodes of the next level below.

Logarithmic search time. We will perform a backward analysis. Suppose that we are searching for an item x in a skip list, and imaging that we are following the path backward from x to the start point of the search, *i.e.*, the $-\infty$ node in the topmost level of the skip list. By "imagine", we mean that a skip list does not have left or up pointers that really allow us to go backward: we are just doing this for analysis purpose. Now each edge in this path has one of the two possible types. Either it is an edge between two copies in adjacent levels (up), or it is an edge between a node and its predecessor in the same level (left). The second type of edge occurs only if the item does not have a copy at its next level up (*i.e.*, when there are edges both up and to the left, we will go left).

Now, we use deferred decision making to bound the expected length of the search path. Imagine we are building the skip list as the search proceeds in reverse as follows. When at an item w in level l, we toss a coin to determine whether there is a copy of w in level $l+1$. In addition, once the search path has moved up to level l, we perform coin tosses for all the remaining items on level l to determine whether they have copies at level $l+1$. Since heads (copy) and tails (no copy) are equally likely, so long as the $-\infty$ item has not been reached, the expected number of coin tosses to go up a level is 2 (since this is a geometric random variable with parameter 1/2). Once the $-\infty$ item is reached, no more coin tosses are needed: the rest of the reverse search path simply goes up level by level.

Let L be the topmost level with a copy of a real item in it (*i.e.*, the top level with only the $\pm\infty$ items is level $L+1$). Therefore, the expected number of coin tosses to reach level $\min\{1+\log n, L\}$ is at most $2(1+\log n)$. Now note that there are i items at level $1+\log n$ with probability at most

$$\binom{n}{i}\frac{1}{2^{i(1+\log n)}} \le \frac{n^i}{i!}\cdot\frac{1}{2^i n^i} \le \frac{1}{2^i\cdot i!}. \tag{49}$$

We will show inductively the following bound $C(n)$ on the expected number of coin tosses for a search in a skip list of n items, such that

$$C(n) = \begin{cases} 2, & \text{if } n=1, \\ 2\log 2 + 4 & \text{if } n=2, \\ 2\lceil \log n\rceil + 5, & \text{if } n\ge 3. \end{cases} \tag{50}$$

3/1 Lecture

To verify this bound, for $n=1$, the number of coin tosses is the number of levels that need to be climbed, which is 1 in expectation, and equals the number of heads. The number of tails is exactly 1. This gives a total of 2 coin tosses in expectation. As for $n\ge 2$, we start by looking at the situation after $1+\lceil\log n\rceil$ heads are performed. At this point, there are i items present at level $1+\lceil\log n\rceil$ with probability at most $\frac{1}{2^i\cdot i!}$. We can view the cost of the search starting at this level as being the same as the cost of a search on a skip list of i items. For $n=2$, we first go up $1+\log 2$ levels, with each level taking 2 tosses in expectation. Then there is 1/2 probability that only 1 item is present on this level, and another 1/8 probability that both 2 items are present on this level. This yields the bound

$$C(2) \le 2(1+\log 2) + \frac{C(1)}{2} + \frac{C(2)}{8} = 3 + 2\log 2 + \frac{C(2)}{8} \implies C(2) \le \frac{8}{7}(3+2\log 2) \le 2\log 2 + 4. \tag{51}$$

Similarly, for $n\ge 3$, we can deduce that

$$\begin{aligned} C(n) &\le 2(1+\lceil\log n\rceil) + \frac{C(1)}{2} + \frac{C(2)}{8} + \sum_{i=3}^{n}\frac{C(i)}{2^i\cdot i!} \le 2(1+\lceil\log n\rceil) + 1 + \frac{3}{4} + \underbrace{\sum_{i=3}^{n}\frac{2\lceil\log n\rceil+5}{2^i\cdot i!}}_{\text{induction hypothesis}} \\ &\le 2\lceil\log n\rceil + \frac{15}{4} + \underbrace{2\cdot\frac{2\log 4+5}{2^3\cdot 3!}}_{i=3,4} + \underbrace{4\cdot\frac{2\log 8+5}{2^5\cdot 5!}}_{i=5,6,7,8} + \cdots \le 2\lceil\log n\rceil + \frac{15}{4} + \sum_{h=1}^{\infty}\left(2^h\cdot\frac{2(h+1)+5}{2^{2^h+1}(2^h+1)!}\right). \end{aligned} \tag{52}$$

This is upper bounded by $2\lceil\log n\rceil + 5$. Any rigorous proof?

4 Moments and Deviations

4.1 Markov's and Chebyshev's Inequalities

Theorem 4.1 (Markov's inequality)**.** Let X be a random variable that assumes only nonnegative values. Then for all $a > 0$, we have that

$$\mathbb{P}[X \geq a] \leq \frac{\mathbb{E}[X]}{a}. \tag{53}$$

Proof. This is trivial by definition, since

$$\mathbb{E}[X] = \sum_x x\mathbb{P}[X = x] \geq \sum_{x \geq a} x\mathbb{P}[X = x] \geq a \sum_{x \geq a} \mathbb{P}[X = x] = a\mathbb{P}[X \geq a], \tag{54}$$

which by dividing both sides by a, we can conclude the desired result. □

The kth **moment** of a random variable X is defined as $\mathbb{E}[X^k]$. The **variance** of X is then defined as

$$\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}^2[X]. \tag{55}$$

The **covariance** of two random variables X and Y is defined by

$$\mathrm{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]. \tag{56}$$

Theorem 4.2. For any two random variables X and Y, we have that

$$\mathrm{Var}[X + Y] = \mathrm{Var}[X] + \mathrm{Var}[Y] + 2\mathrm{Cov}[X, Y]. \tag{57}$$

Proof. The proof is trivial by definition of variance and linearity of expectation, and will be ignored here. □

Theorem 4.3. If X and Y are two independent random variables, then

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]. \tag{58}$$

Proof. By definition of expectation, we have that

$$\begin{aligned}\mathbb{E}[XY] &= \sum_x \sum_y xy\mathbb{P}[(X = x) \wedge (Y = y)] = \sum_x \sum_y xy \underbrace{\mathbb{P}[X = x]\mathbb{P}[Y = y]}_{\text{by independence}} \\ &= \sum_x x\mathbb{P}[X = x] \left(\sum_y y\mathbb{P}[Y = y] \right) = \mathbb{E}[Y] \sum_x x\mathbb{P}[X = x] = \mathbb{E}[X]\mathbb{E}[Y],\end{aligned} \tag{59}$$

as desired, so the proof is complete. □

Note that if X and Y are independent random variables, we also have that

$$\mathrm{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[X - \mathbb{E}[X]]\mathbb{E}[Y - \mathbb{E}[Y]] = (\mathbb{E}[X] - \mathbb{E}[X])(\mathbb{E}[Y] - \mathbb{E}[Y]) = 0, \tag{60}$$

and

$$\mathrm{Var}[X + Y] = \mathrm{Var}[X] + \mathrm{Var}[Y] + 2\mathrm{Cov}[X, Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]. \tag{61}$$

The second observation can be generated, so that if $X_1, \cdots, X_n$ are mutually independent random variables, then

$$\mathrm{Var}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathrm{Var}[X_i]. \tag{62}$$

For instance, the variance of a Bernoulli random variable X with parameter p is

$$\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = p \cdot (1 - p)^2 + (1 - p) \cdot (0 - p)^2 = p(1 - p)(1 - p + p) = p(1 - p), \tag{63}$$

and the variance of a binomial random variable Y as the sum of n such Bernoulli random variables is

$$\mathrm{Var}[Y] = np(1 - p). \tag{64}$$

Theorem 4.4 (Chebyshev's inequality)**.** For any $a > 0$, we have that

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq a] \leq \frac{\mathrm{Var}[X]}{a^2}. \tag{65}$$

Proof. By Markov's inequality, we can deduce that

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq a] = \mathbb{P}[(X - \mathbb{E}[X])^2 \geq a^2] \leq \frac{\mathbb{E}[(X - \mathbb{E}[X])^2]}{a^2} = \frac{\mathrm{Var}[X]}{a^2}, \tag{66}$$

as desired, so the proof is complete. □

Last Modified: Mar 3, 2023.